

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/382632835>

Lifelong Event Detection with Embedding Space Separation and Compaction

Conference Paper · January 2024

DOI: 10.18653/v1/2024.naacl-short.50

CITATIONS

2

READ

1

5 authors, including:



Chengwei Qin

Nanyang Technological University

37 PUBLICATIONS 1,053 CITATIONS

SEE PROFILE



Wenhan Xia

Princeton University

15 PUBLICATIONS 123 CITATIONS

SEE PROFILE



Shafiq Joty

University of British Columbia

205 PUBLICATIONS 5,434 CITATIONS

SEE PROFILE

Lifelong Event Detection with Embedding Space Separation and Compaction

Chengwei Qin[†], Ruirui Chen[♦], Ruochen Zhao[†], Wenhan Xia[♦], Shafiq Joty^{†♦}

[†]Nanyang Technological University [♦]Princeton University [♦]Salesforce Research

[♦]Institute of High Performance Computing (IHPC),

Agency for Science, Technology and Research (A*STAR)

1 Fusionopolis Way, #16-16 Connexis, Singapore 138632, Republic of Singapore

{chengwei003@e.ntu, ruochen002@e.ntu, srjoty@ntu }.edu.sg

chen_ruirui@ihpc.a-star.edu.sg, wxia@princeton.edu

Abstract

To mitigate forgetting, existing lifelong event detection methods typically maintain a memory module and replay the stored memory data during the learning of a new task. However, the simple combination of memory data and new-task samples can still result in substantial forgetting of previously acquired knowledge, which may occur due to the potential overlap between the feature distribution of new data and the previously learned embedding space. Moreover, the model suffers from overfitting on the few memory samples rather than effectively remembering learned patterns. To address the challenges of forgetting and overfitting, we propose a novel method based on embedding space separation and compaction. Our method alleviates forgetting of previously learned tasks by forcing the feature distribution of new data away from the previous embedding space. It also mitigates overfitting by a memory calibration mechanism that encourages memory data to be close to its prototype to enhance intra-class compactness. In addition, the learnable parameters of the new task are initialized by drawing upon acquired knowledge from the previously learned task to facilitate forward knowledge transfer. With extensive experiments, we demonstrate that our method can significantly outperform previous state-of-the-art approaches.

1 Introduction

Event detection (ED) aims to detect the event type of trigger words in a given sentence, *e.g.*, extracting the event type *injure* from the trigger word *scalded* in text “*He was scalded by hot water*”. Traditional ED methods typically consider a fixed pre-defined set of event types (Chen et al., 2015; Nguyen et al., 2016; Huang and Ji, 2020; Chen et al., 2024). However, as the environment and data distributions change in real scenarios, the model might face challenges in handling rapidly emerging new types (Lu et al., 2022).

A more practical setting is lifelong event detection or LED (Cao et al., 2020), where the model learns event knowledge from a sequence of tasks with different sets of event types. In LED, the model is expected to retain and accumulate knowledge when learning new tasks, which is challenging due to *catastrophic forgetting* (McCloskey and Cohen, 1989) of previously acquired knowledge. Existing methods (Cao et al., 2020; Yu et al., 2021) for mitigating forgetting in LED typically maintain a memory that saves a few key samples of previous tasks, which are then combined with new data for training. Recently, Liu et al. (2022) introduce Episodic Memory Prompts (EMP) that leverages soft prompts to remember learned event types, achieving state-of-the-art performance on LED.

Despite its effectiveness, EMP has two key limitations. First, simply combining new data and memory samples for training can still result in forgetting as the feature distribution of new data might overlap with the previously learned embedding space (see Appendix A.1). Second, it may overfit on a few memory samples after frequent replays rather than effectively retaining learned patterns.

To address the above limitations of EMP, in this paper, we introduce a novel method based on Embedding space Separation and COmpaction (ESCO) for LED. In particular, we propose a margin-based loss that forces the feature distribution of new event types away from the learned embedding space to alleviate *forgetting*. Inspired by Han et al. (2020), we introduce a memory calibration mechanism to encourage memory data to be close to its prototype to avoid *overfitting* on the few memory samples. In addition, the learnable parameters of the new task are initialized using those of the previously learned task to facilitate *forward knowledge transfer*, which is as important for lifelong learning as preventing forgetting (Ke et al., 2020; Qin et al., 2023a,b). The empirical results show that our method significantly outperforms

previous state-of-the-art approaches. In summary, our main contributions are:

- We propose ESCO, a novel method based on embedding space separation and compaction to mitigate forgetting and overfitting in LED.
- With extensive experiments and analysis, we demonstrate the effectiveness of our method compared to existing ones.

2 Problem Formulation

LED involves learning from a stream of event detection tasks $\mathbb{T} = (\mathcal{T}^1, \dots, \mathcal{T}^n)$, where each task $\mathcal{T}^k$ has its own training set $\mathcal{D}_{\text{train}}^k$, validation set $\mathcal{D}_{\text{valid}}^k$, and test set $\mathcal{D}_{\text{test}}^k$. For every input text x^i in $\mathcal{D}^k$, it contains a set of target spans $\{\bar{x}_t^i\}$ and their corresponding labels y_t^i which belong to the event type set $\mathcal{C}^k$ of task $\mathcal{T}^k$. Note that the event type sets of different tasks are non-overlapping.

After the training on $\mathcal{D}_{\text{train}}^k$, the model is expected to perform well on all the k tasks that it has learned and will be evaluated on the combined test set $\hat{\mathcal{D}}_{\text{test}}^k = \cup_{i=1}^k \mathcal{D}_{\text{test}}^i$ consisting of all known event types $\hat{\mathcal{C}}^k = \cup_{i=1}^k \mathcal{C}^i$. During the learning, a memory module $\mathcal{M}$ which stores a few key samples of previous tasks is maintained to overcome the forgetting problem.

3 Embedding Space Separation and Compaction

When learning a new task $\mathcal{T}^k$, following Liu et al. (2022), we first initialize a set of soft prompts $\mathcal{P}^k = \{p_1^k, \dots, p_{|\mathcal{C}^k|}^k\}$ where $\mathcal{C}^k$ is the event type set of $\mathcal{T}^k$. The accumulated prompts $\mathcal{Q}^k = [\mathcal{P}^1, \dots, \mathcal{P}^k]$ until time step k are then combined with the input text x^i to obtain the contextual representations using a frozen BERT (Devlin et al., 2019):

$$[\mathbf{x}^i, \mathbf{Q}^k] = \text{BERT}([x^i, \mathcal{Q}^k]) \quad (1)$$

where $\mathbf{x}^i$ and $\mathbf{Q}^k$ are the representations of x^i and $\mathcal{Q}^k$, respectively. To facilitate *forward knowledge transfer* (Qin and Joty, 2022b), we initialize soft prompts $\mathcal{P}^k$ of the new task using learned prompts $\mathcal{P}^{k-1}$ of the previous task. For the first task $\mathcal{T}^1$, we initialize each event type prompt p_i^1 in $\mathcal{P}^1$ using its corresponding name.

To predict the event type of the span $\bar{x}_t^i$, we concatenate the representations corresponding to the start and end token and obtain the logits over all learned types through a feed-forward network

(FFN) as well as a linear layer:

$$Z_t^i = \text{Linear}(\text{FFN}([\mathbf{x}_m^i, \mathbf{x}_n^i])) \quad (2)$$

where $\bar{\mathbf{x}}_t^i = \text{FFN}([\mathbf{x}_m^i, \mathbf{x}_n^i])$ is the span representation, m and n denote the start and end index of the span, respectively. Following Liu et al. (2022), to entangle span representations with soft prompts, the probability distribution over all prompts is calculated as $Z_q = \text{FFN}(\mathbf{Q}^k) \cdot \bar{\mathbf{x}}_t^i$, where $\cdot$ is the inner product. Z_q is then combined with Z_t^i to optimize the cross entropy loss:

$$\mathcal{L}_{\text{new}} = - \sum_{(\bar{x}_t^i, y_t^i) \in \mathcal{D}_{\text{train}}^k} \text{CE}(Z_t^i + Z_q, y_t^i) \quad (3)$$

After learning the previous task $\mathcal{T}^{k-1}$, we select the top- l most informative training examples for each event type in $\mathcal{C}^{k-1}$ using the herding algorithm (Welling, 2009), which are then saved in the memory module $\mathcal{M}$ for replay to mitigate forgetting. Similar as Eq. 3, the training objective for memory replay when learning $\mathcal{T}^k$ is:

$$\mathcal{L}_{\text{mem}} = - \sum_{(\bar{x}_t^i, y_t^i) \in \mathcal{M}} \text{CE}(Z_t^i + Z_q, y_t^i) \quad (4)$$

However, the simple combination of $\mathcal{L}_{\text{new}}$ and $\mathcal{L}_{\text{mem}}$ can still result in substantial forgetting of acquired knowledge due to the potential overlap between the feature distribution of new event types and the previously learned embedding space (see Appendix A.1). To ensure that the new feature distribution is away from the learned embedding space, we design a *margin-based* loss, which decreases the similarity scores between new samples and prototypes (see Eq. 8 for the calculation of prototypes) of learned event types:

$$\mathcal{L}_{\text{sim}} = \sum_{(\bar{x}_t^i, y_t^i) \in \mathcal{D}_{\text{train}}^k} \sum_{\mathbf{e}_i \in \mathcal{E}^{k-1}} \max(0, g(\bar{\mathbf{x}}_t^i, \mathbf{e}_i) - m_1) \quad (5)$$

where $\mathcal{E}^{k-1}$ is the prototype set of previous $k-1$ tasks, $g(\cdot)$ is the similarity function (cosine similarity) and m_1 is the margin for $\mathcal{L}_{\text{sim}}$. Note that $\mathcal{L}_{\text{sim}}$ is different from metric learning or contrastive learning (Qin and Joty, 2022a) which typically considers both positive and negative pairs. $\mathcal{L}_{\text{sim}}$ only includes negative pairs while ignoring positive ones as our goal in designing $\mathcal{L}_{\text{sim}}$ is to *separate* the new feature distribution and the learned embedding space.

As the size of memory $\mathcal{M}$ is typically small, the model is prone to overfit on the few memory samples after frequent replays, making learned distributions distorted. To effectively recover from distorted learned distributions, we introduce a *memory calibration* mechanism inspired by Han et al. (2020). Specifically, for each memory sample in $\mathcal{M}$, we encourage it to be close to its corresponding prototype to improve the intra-class compactness of learned distributions. More formally,

$$\mathcal{L}_{\text{cal}} = - \sum_{(\bar{\mathbf{x}}_t^i, y_t^i) \in \mathcal{M}} \log \frac{\exp g(\bar{\mathbf{x}}_t^i, \mathbf{e}_t)}{\sum_{j=1}^{|\mathcal{E}^{k-1}|} \exp g(\bar{\mathbf{x}}_t^i, \mathbf{e}_j)} \quad (6)$$

where $\mathbf{e}_t$ is the prototype of y_t^i . The *total* loss for learning on $\mathcal{T}^k$ is defined as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{new}} + \lambda_1 \mathcal{L}_{\text{sim}} + \lambda_2 (\mathcal{L}_{\text{mem}} + \mathcal{L}_{\text{cal}}) \quad (7)$$

where λ_1 and λ_2 are loss weights.

After learning $\mathcal{T}^k$ and selecting memory data for $\mathcal{T}^k$, we use the memory $\mathcal{M}$ to calculate prototypes of all learned event types in $\mathcal{C}^k$. Specifically, for each event type e_j in $\mathcal{C}^k$, we obtain its prototype $\mathbf{e}_j$ by averaging the span representations of all samples labeled as e_j in $\mathcal{M}$ as follows:

$$\mathbf{e}_j = \frac{1}{|\mathcal{M}_{e_j}|} \sum_{(\bar{\mathbf{x}}_t^i, y_t^i) \in \mathcal{M}_{e_j}} \bar{\mathbf{x}}_t^i \quad (8)$$

where $\mathcal{M}_{e_j} = \{(\bar{\mathbf{x}}_t^i, y_t^i) | (\bar{\mathbf{x}}_t^i, y_t^i) \in \mathcal{M}, y_t^i = e_j\}$.

4 Experiment

4.1 Experimental Setup

We conduct experiments on two representative event detection datasets in our work: ACE05 (Doddington et al., 2004) and MAVEN (Wang et al., 2020). Following Liu et al. (2022), we divide each dataset into a sequence of 5 tasks with non-overlapping event type sets to form a *class-incremental* setting. After learning $\mathcal{T}^k$, the model is evaluated on the combined test set $\hat{\mathcal{D}}_{\text{test}}^k = \bigcup_{i=1}^k \mathcal{D}_{\text{test}}^i$ of all seen tasks. As the task order might influence the model performance, we run experiments for each dataset 5 times with different task order permutations and report the average results. More details of the experimental setup are in Appendix A.2.

4.2 Methods Compared

We compare our approach with the following methods: (1) **Fine-tuning** tunes the model only on new data without memory; (2) **BiC** (Wu et al., 2019) introduces a bias correction layer to improve lifelong learning performance; (3) **KCN** (Cao et al., 2020) designs prototype enhanced retrospection and hierarchical distillation to alleviate semantic ambiguity and class imbalance; (4) **KT** (Yu et al., 2021) proposes to transfer knowledge between related types; (5) **EMP** (Liu et al., 2022) leverages type-specific soft prompts to remember learned event knowledge; and (6) **Multi-task learning (MTL)** simultaneously trains the model on all data, serving as the *upper bound* in LED.

4.3 Main Results

We report the F1 score of different methods at each time step in Table 1. From the results, we can observe that ESCO significantly outperforms previous baselines on both datasets, demonstrating its superiority. Simply fine-tuning the model on new data without memory replay results in poor performance due to severe forgetting of learned knowledge. Although BiC, KCN and KT could alleviate forgetting to some extent, there is still a large performance drop after learning all tasks. EMP achieves better performance because the type-specific soft prompts help retain previously acquired knowledge. However, it does not necessarily ensure large distances among feature distributions of different event types, and easily overfits on the memory samples. Our proposed ESCO outperforms EMP by a large margin through embedding space separation and compaction. To verify its effectiveness, we visualize the embedding spaces of EMP and ESCO on ACE05 in Fig. 1. Specifically, we randomly select 6 event types from different learning stages and visualize their test data using t-SNE (Van der Maaten and Hinton, 2008). The comparison demonstrates that ESCO could achieve larger inter-class distances and better intra-class compactness in the embedding space.

4.4 Ablation Study

To analyze the contribution of different components of ESCO, we conduct several ablations. Specifically, we investigate three variants of ESCO: (a) without the margin-based loss (*w.o.* margin), (b) removing the memory calibration mechanism (*w.o.* calibration), and (c) without forward knowl-

	MAVEN					ACE05				
Task index	1	2	3	4	5	1	2	3	4	5
Fine-tuning	63.51	39.99	33.36	23.83	22.69	58.30	43.96	38.02	21.53	25.71
BiC	63.51	46.69	39.15	31.69	30.47	58.30	45.73	43.28	35.70	30.80
KCN	63.51	51.17	46.80	38.72	38.58	58.30	54.71	52.88	44.93	41.10
KT	63.51	52.36	47.24	39.51	39.34	58.30	55.41	53.95	45.00	42.62
EMP	67.50	59.67	58.03	54.80	54.39	58.35	50.03	54.91	47.78	47.19
ESCO	67.50	61.37	60.65	57.43	57.35	58.35	57.42	57.63	53.64	55.20
MTL	—	—	—	—	68.42	—	—	—	—	67.22

Table 1: F1 score (%) of different methods at every time step on two datasets. ‘MTL’ stands for ‘multi-task learning’. ESCO is significantly better than EMP with p -value < 0.05 (paired t-test). We report results with variance and detailed results for different task orders in Appendix A.3 and Appendix A.4, respectively.

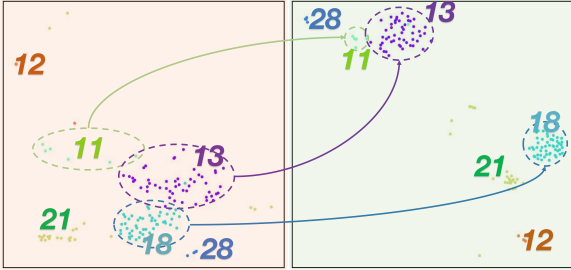


Figure 1: Comparison between the embedding spaces of EMP (left) and ESCO (right). Colors represent different event types with numbers being the event indexes. Compared with EMP, ESCO shows larger inter-class distances, e.g., the distance between 13 and 18, and better intra-class compactness (circled regions).

edge transfer (FKT) (*w.o.* FKT). The results of different ablations after learning all tasks are reported in Table 2. We can see that all components contribute to overall performance. The margin-based loss yields about **1.25%** performance boost as it can bring feature distribution of new data away from the learned embedding space. The memory calibration mechanism improves the F1 score by **1.47%**, which demonstrates the necessity of improving intra-class compactness of learned distributions. The adoption of forward knowledge transfer leads to **0.80%** improvement, indicating that it can indeed transfer useful learned knowledge to facilitate the learning of new tasks.

4.5 Further Analysis

Quantify Knowledge Transfer. Following Lopez-Paz and Ranzato (2017), we report the backward transfer (BWT) and forward transfer (FWT) of EMP and ESCO after learning all tasks

Method	MAVEN	ACE05	Average
ESCO	57.35	55.20	56.28
<i>w.o.</i> margin	55.92	54.13	55.03
<i>w.o.</i> calibration	55.76	53.85	54.81
<i>w.o.</i> FKT	56.58	54.38	55.48

Table 2: F1 score (%) of different ablations after learning all tasks: (i) without the margin-based loss, (ii) without the memory calibration mechanism, and (iii) without forward knowledge transfer. All components improve the performance of our method.

Dataset	MAVEN		ACE05	
	BWT	FWT	BWT	FWT
EMP	-10.4	-2.9	-16.8	0.5
ESCO	-6.3	-1.2	-7.5	4.1

Table 3: Backward transfer (BWT) and forward transfer (FWT) of EMP and ESCO after learning all tasks on MAVEN and ACE05.

in Table 3. From the comparison, we can observe that ESCO outperforms EMP by a large margin in terms of BWT and FWT on both datasets, demonstrating its effectiveness.

In addition, we show results of a different backbone model (RoBERTa (Liu et al., 2019)), the effect of memory size, results of different memory sample selection methods, the comparison with the contrastive loss in Qin and Joty (2022a), and a case study of the model output in Appendix A.5 ~ A.9, respectively.

5 Related Work

Lifelong event detection (LED) aims to continually learn from a sequence of event detection tasks

with different sets of event types. Cao et al. (2020) propose KCN which addresses the semantic ambiguity and data imbalance problems in LED by prototype enhanced retrospection and hierarchical distillation. KT (Yu et al., 2021) encourages bi-directional knowledge transfer between old and new event types. Liu et al. (2022) introduce EMP to retain previously learned task-specific event knowledge through soft prompts. In contrast to previous works, we innovate on the methodology by imposing further constraints in the embedding space to mitigate forgetting and overfitting.

6 Conclusion

In this work, we have introduced embedding space separation and compaction (ESCO) for lifelong event detection (LED). ESCO imposes novel feature constraints in the embedding space to alleviate forgetting and overfitting problems. It initializes the learnable parameters for the new task by inheriting those from the previously learned task to facilitate forward knowledge transfer. With extensive experiments and analysis, we have demonstrated that ESCO significantly outperforms previous methods. For future work, we are interested in exploring ESCO in a meta-learning paradigm for LED.

Limitations

Although effective, ESCO also has some limitations. For example, ESCO mainly focuses on the setting where each task has enough training data. Few-shot learning and in-context learning have been receiving increasing interest within the community. We leave how to explore lifelong event detection in few-shot settings as future work. Besides, as large language models (LLMs) have shown impressive performance on a variety of tasks (Brown et al., 2020; Ouyang et al., 2022; Touvron et al., 2023; Qin et al., 2023c,d,e; Achiam et al., 2023; Xia et al., 2024; Ding et al., 2023), a further improvement could be to explore lifelong event detection with LLMs.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. *Gpt-4 technical report*. *arXiv preprint arXiv:2303.08774*.

Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind

Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. *Language models are few-shot learners*. *arXiv preprint arXiv:2005.14165*.

Pengfei Cao, Yubo Chen, Jun Zhao, and Taifeng Wang. 2020. *Incremental event detection via knowledge consolidation networks*. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 707–717, Online. Association for Computational Linguistics.

Ruirui Chen, Chengwei Qin, Weifeng Jiang, and Dongkyu Choi. 2024. *Is a large language model a good annotator for event extraction?* In *Proceedings of the AAAI Conference on Artificial Intelligence*, 16, pages 17772–17780.

Yubo Chen, Liheng Xu, Kang Liu, Daojian Zeng, and Jun Zhao. 2015. *Event extraction via dynamic multi-pooling convolutional neural networks*. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 167–176, Beijing, China. Association for Computational Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. *BERT: Pre-training of deep bidirectional transformers for language understanding*. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Bosheng Ding, Chengwei Qin, Linlin Liu, Yew Ken Chia, Boyang Li, Shafiq Joty, and Lidong Bing. 2023. *Is GPT-3 a good data annotator?* In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11173–11195, Toronto, Canada. Association for Computational Linguistics.

George Doddington, Alexis Mitchell, Mark Przybocki, Lance Ramshaw, Stephanie Strassel, and Ralph Weischedel. 2004. *The automatic content extraction (ACE) program – tasks, data, and evaluation*. In *Proceedings of the Fourth International Conference on Language Resources and Evaluation (LREC’04)*, Lisbon, Portugal. European Language Resources Association (ELRA).

Xu Han, Yi Dai, Tianyu Gao, Yankai Lin, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. 2020. *Continuous relation learning via episodic memory activation and reconsolidation*. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6429–6440, Online. Association for Computational Linguistics.

Lifu Huang and Heng Ji. 2020. *Semi-supervised new event type induction and event detection*. In *Proceedings of the 2020 Conference on Empirical Methods*

- in *Natural Language Processing (EMNLP)*, pages 718–724, Online. Association for Computational Linguistics.
- Zixuan Ke, Bing Liu, and Xingchang Huang. 2020. [Continual learning of a mixed sequence of similar and dissimilar tasks](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 18493–18504. Curran Associates, Inc.
- Minqian Liu, Shiyu Chang, and Lifu Huang. 2022. [Incremental prompting: Episodic memory prompt for lifelong event detection](#). In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 2157–2165, Gyeongju, Republic of Korea. International Committee on Computational Linguistics.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. [Roberta: A robustly optimized bert pretraining approach](#). *arXiv preprint arXiv:1907.11692*.
- David Lopez-Paz and Marc’Aurelio Ranzato. 2017. [Gradient episodic memory for continual learning](#). *Advances in neural information processing systems*, 30.
- Zhou Lu, Wenhan Xia, Sanjeev Arora, and Elad Hazan. 2022. Adaptive gradient methods with local guarantees. *arXiv preprint arXiv:2203.01400*.
- Michael McCloskey and Neal J Cohen. 1989. [Catastrophic interference in connectionist networks: The sequential learning problem](#). In *Psychology of learning and motivation*, volume 24, pages 109–165. Elsevier.
- Thien Huu Nguyen, Kyunghyun Cho, and Ralph Grishman. 2016. [Joint event extraction via recurrent neural networks](#). In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 300–309, San Diego, California. Association for Computational Linguistics.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. [Training language models to follow instructions with human feedback](#). *Advances in Neural Information Processing Systems*, 35:27730–27744.
- Chengwei Qin and Shafiq Joty. 2022a. [Continual few-shot relation learning via embedding space regularization and data augmentation](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2776–2789, Dublin, Ireland. Association for Computational Linguistics.
- Chengwei Qin and Shafiq Joty. 2022b. [LFPT5: A unified framework for lifelong few-shot language learning based on prompt tuning of t5](#). In *International Conference on Learning Representations*.
- Chengwei Qin, Shafiq Joty, and Chen Chen. 2023a. [Lifelong sequence generation with dynamic module expansion and adaptation](#). *arXiv preprint arXiv:2310.09886*.
- Chengwei Qin, Shafiq Joty, Qian Li, and Ruochen Zhao. 2023b. [Learning to initialize: Can meta learning improve cross-task generalization in prompt tuning?](#) In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11802–11832, Toronto, Canada. Association for Computational Linguistics.
- Chengwei Qin, Wenhan Xia, Fangkai Jiao, and Shafiq Joty. 2023c. [Improving in-context learning via bidirectional alignment](#). *arXiv preprint arXiv:2312.17055*.
- Chengwei Qin, Aston Zhang, Anirudh Dagar, and Wenming Ye. 2023d. [In-context learning with iterative demonstration selection](#). *arXiv preprint arXiv:2310.09881*.
- Chengwei Qin, Aston Zhang, Zhuosheng Zhang, Jiao Chen, Michihiro Yasunaga, and Diyi Yang. 2023e. [Is chatgpt a general-purpose natural language processing task solver?](#) *arXiv preprint arXiv:2302.06476*.
- Qing Sun, Fan Lyu, Fanhua Shang, Wei Feng, and Liang Wan. 2022. [Exploring example influence in continual learning](#). In *Advances in Neural Information Processing Systems*.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. [Llama 2: Open foundation and fine-tuned chat models](#). *arXiv preprint arXiv:2307.09288*.
- Laurens Van der Maaten and Geoffrey Hinton. 2008. [Visualizing data using t-sne](#). *Journal of machine learning research*, 9(11).
- Xiaozhi Wang, Ziqi Wang, Xu Han, Wangyi Jiang, Rong Han, Zhiyuan Liu, Juanzi Li, Peng Li, Yankai Lin, and Jie Zhou. 2020. [MAVEN: A Massive General Domain Event Detection Dataset](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1652–1671, Online. Association for Computational Linguistics.
- Max Welling. 2009. [Herding dynamical weights to learn](#). In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 1121–1128.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. [Trans-formers: State-of-the-art natural language processing](#).

	MAVEN				
Task index	1	2	3	4	5
EMP	67.1	58.3	55.7	53.2	52.9
ESCO	67.1	60.8	59.0	55.3	55.1

Table 4: Performance comparison between EMP and ESCO on MAVEN using RoBERTa as the backbone.

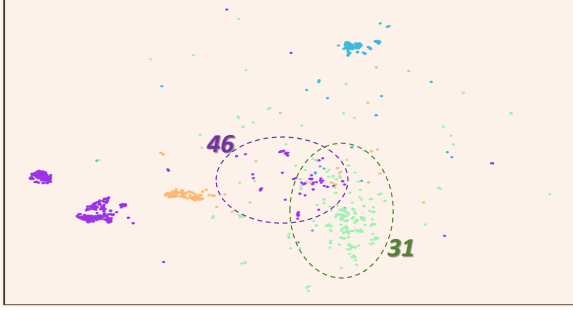


Figure 2: Overlap between feature distributions of event types at different learning stages, *e.g.*, circled regions. Colors represent different event types with numbers being the event indexes.

In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Yue Wu, Yinpeng Chen, Lijuan Wang, Yuancheng Ye, Zicheng Liu, Yandong Guo, and Yun Fu. 2019. [Large scale incremental learning](#). In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 374–382.

Wenhan Xia, Chengwei Qin, and Elad Hazan. 2024. Chain of lora: Efficient fine-tuning of language models via residual learning. *arXiv preprint arXiv:2401.04151*.

Pengfei Yu, Heng Ji, and Prem Natarajan. 2021. [Life-long event detection with knowledge transfer](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5278–5290, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

A Appendix

A.1 Overlap of Feature Distributions

Fig. 2 shows an example of the overlap between feature distributions of event types at different learning stages.

A.2 Implementation Details

All methods are implemented with PyTorch/Transformers library (Wolf et al., 2020). For hyperparameters, we mainly follow the settings

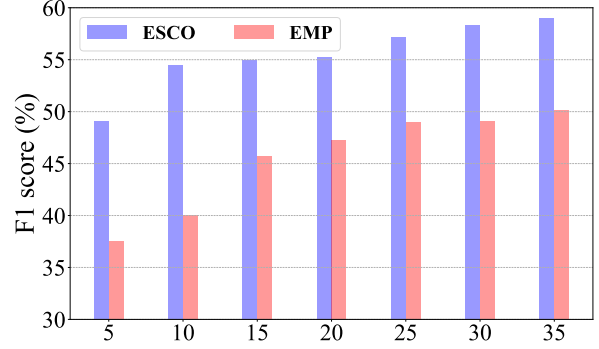


Figure 3: The performance of ESCO and EMP with different memory sizes.

in Liu et al. (2022) to have a fair comparison. We adopt -0.1 for the margin value m_1 so that the similarity score between a new sample and the prototype of a previously learned event type could be optimized to a negative number, *i.e.*, large inter-class distance. We set the weight λ_1 to 0.1 so that its corresponding loss $\mathcal{L}_{\text{sim}}$ has roughly the same order of magnitude as other losses. The loss weight λ_2 is set to $\frac{s}{k+s}$, where k is the number of target spans in the current batch and s is equal to 50 following Liu et al. (2022). For each task, we train the model for 20 epochs with early stopping.

For the state-of-the-art EMP (Liu et al., 2022), we reproduce the results using its open-source code and the same environment. For our method, we use the same environment and shared hyperparameters as EMP. For other baselines, we reuse the results in Liu et al. (2022). There are mainly two reasons: (a) They perform much worse than EMP, *i.e.*, they are not primary comparison approaches in our work; and (b) EMP reports different baseline results from Yu et al. (2021), indicating different settings. However, EMP does not provide details on how to obtain baseline results. As we use the same setting as EMP, we decide to reuse its results for other baselines.

A.3 Results with Variance

We show results with variance for EMP and ESCO in Table 5.

A.4 Detailed Results for Different Task Orders

Table 6 reports detailed results for different task orders.

	MAVEN					ACE05				
Task index	1	2	3	4	5	1	2	3	4	5
EMP	67.50 _{±3.54}	59.67 _{±2.74}	58.03 _{±1.44}	54.80 _{±0.95}	54.39 _{±0.82}	58.35 _{±6.92}	50.03 _{±18.18}	54.91 _{±9.19}	47.78 _{±2.57}	47.19 _{±8.53}
ESCO	67.50 _{±3.54}	61.37 _{±2.92}	60.65 _{±1.85}	57.43 _{±0.81}	57.35 _{±0.66}	58.35 _{±6.92}	57.42 _{±13.56}	57.63 _{±6.26}	53.64 _{±4.41}	55.20 _{±4.16}

Table 5: F1 score (%) and variance of EMP and ESCO at every time step on two datasets.

Task Order	MAVEN					ACE05				
	1	2	3	4	5	1	2	3	4	5
1	70.80	62.19	60.00	56.11	54.95	62.58	65.60	67.20	45.06	39.07
	70.80	64.73	62.22	57.26	57.67	62.58	68.35	66.02	54.13	49.33
2	66.06	56.01	56.16	54.07	54.91	49.17	50.99	57.78	51.76	42.49
	66.06	58.36	58.16	57.71	57.40	49.17	52.13	60.73	57.85	54.76
3	70.80	59.91	57.43	55.53	55.02	62.58	59.43	50.08	46.27	55.12
	70.80	62.70	60.03	58.27	57.57	62.58	61.03	51.96	47.95	59.97
4	67.42	62.35	58.74	54.10	53.19	64.68	55.15	56.87	47.30	41.65
	67.42	62.81	62.72	56.12	56.20	64.68	69.12	58.44	57.78	53.67
5	62.39	57.90	57.81	54.20	53.88	52.74	18.99	42.62	48.53	57.64
	62.39	58.22	60.09	57.78	57.88	52.74	36.49	51.01	50.46	58.29

Table 6: F1 score (%) of 5 runs with different task orders on two datasets. For every order, the upper row shows the performance of EMP and the lower row is the result of ESCO.

	EMP	ESCO
Herding algorithm	54.4	57.1
Example influence	53.5	56.4

Table 7: F1 score (%) of EMP and ESCO with different memory sample selection approaches.

	ESCO	ESCO _{con}
F1 score (%)	57.1	56.6

Table 8: Performance comparison between ESCO and ESCO_{con}.

A.5 Different Backbone Model

To investigate the generalization ability of ESCO, we further conduct experiments on MAVEN using RoBERTa (Liu et al., 2019) backbone. The F1 scores of EMP and ESCO at each time step are reported in Table 4, which verify that ESCO can indeed generalize to different models.

A.6 Effect of Memory Size

Following Liu et al. (2022), we select 20 samples as memory data for each event type. To investigate whether different memory sizes influence the performance gain of ESCO, we conduct controlled experiments on ACE05 with memory size {5, 10, 15, 25, 30, 35}. The performance compari-

son between ESCO and EMP is shown in Fig. 3. We can observe that ESCO consistently outperforms EMP by a large margin with different memory sizes, demonstrating its robustness.

A.7 Different Memory Sample Selection Approaches

Following EMP (Liu et al., 2022), we use the herding algorithm (Welling, 2009) to select memory samples. To validate whether different memory sample selection approaches influence the performance gain of ESCO, we replace the herding algorithm of EMP and ESCO with *example influence* (Sun et al., 2022) for memory selection. We randomly select three sequences for experiments and report the performance comparison between EMP and ESCO in Table 7. We can see that: (a) ESCO consistently outperforms EMP in different cases, demonstrating its effectiveness; and (b) herding algorithm performs better than example influence, justifying our choice.

A.8 Comparison with the Contrastive Loss

As mentioned in §3, our designed margin-based loss $\mathcal{L}_{\text{sim}}$ is different from the contrastive loss as $\mathcal{L}_{\text{sim}}$ only includes negative pairs while ignoring positive ones. To further demonstrate its superiority, we replace it with the contrastive loss in (Qin

Fighting continued until 9 May, when the Red Army entered the nearly liberated city.	
Label	<i>Arriving</i>
EMP	<i>Becoming_a_member</i>
ESCO	<i>Arriving</i>
In Japan, hundreds of people evacuated from mudslide-prone areas.	
Label	<i>Escaping</i>
EMP	<i>Removing</i>
ESCO	<i>Escaping</i>

Table 9: Output examples of EMP and ESCO. We color target spans in blue, correct outputs in green, and wrong outputs in red.

and Joty, 2022a), namely ESCO_{con} . We use the same sequences as Appendix A.7 for experiments and report the results of ESCO and ESCO_{con} in Table 8, which verify the effectiveness of $\mathcal{L}_{\text{sim}}$.

A.9 Case Study

We select MAVEN as a representative task and show several example outputs in Table 9. Compared with EMP, ESCO is able to retain more precise and fine-grained event knowledge, *e.g.*, ESCO can successfully detect the event type *Escaping* from the target span *evacuated* while EMP is confused by another semantically similar type *Removing*.